

Обучение с подкреплением

ШАД, Рекомендательные системы
2025/26, лекция 11

Кирилл Хрыльченко

Старший преподаватель,
ФКН ВШЭ

План занятия

1. Intro to bandits
2. Bandit algorithms
3. Practical considerations
4. Beyond on-policy bandits

На семинаре Руслан разберёт практическое применение бандитных алгоритмов на примере Яндекс Лавки.

Intro to RL bandits

Часть 2

Feedback loop

Repeat:

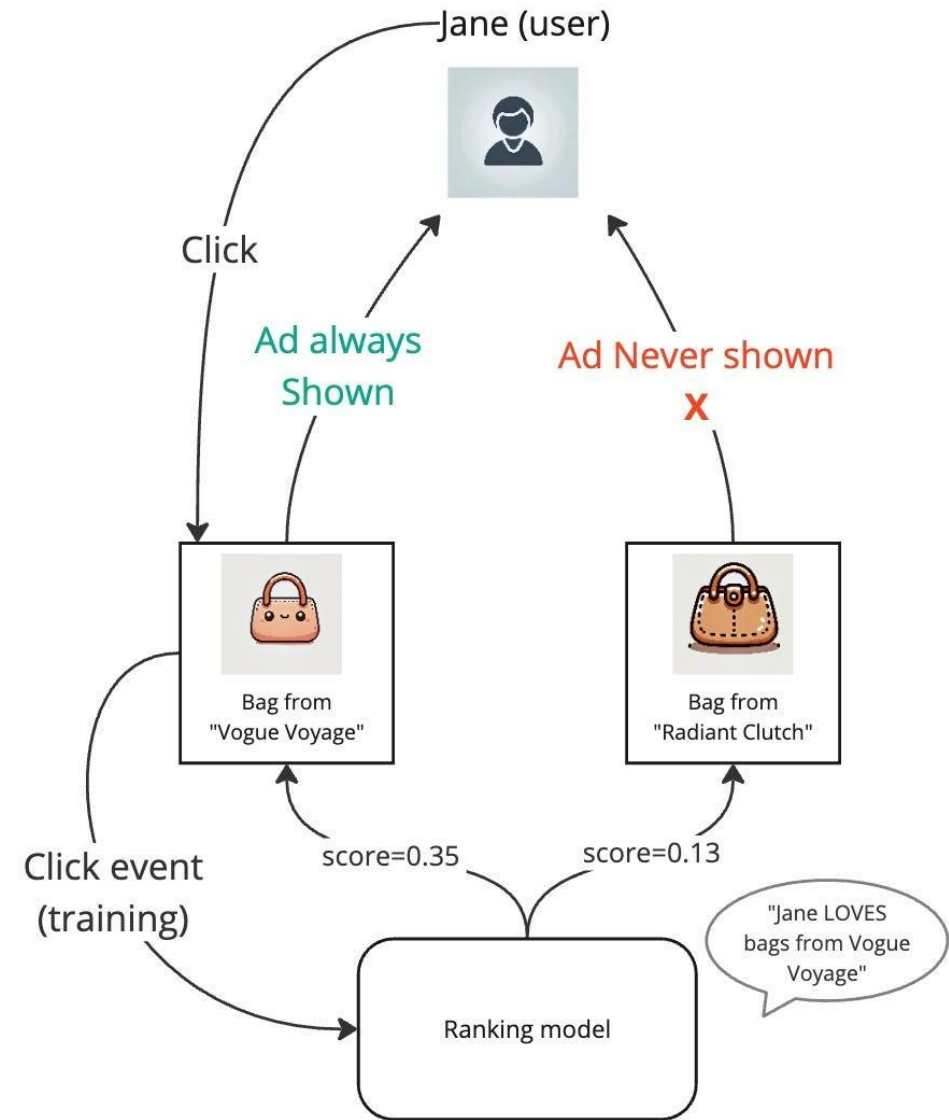
1. Рексистема что-то показала пользователю
2. Пользователь оставил фидбек
3. Рексистема обновилась

Действия системы влияют на будущие действия системы через фидбек.



Exposure bias

1. Рексистема может показать только небольшое подмножество всего доступного контента
2. Пользователь оставляет фидбек только для этого подмножества
3. Рексистема дальше настраивается только на фидбеке для тех айтемов, которые была склонна выдавать и раньше



Bias amplification

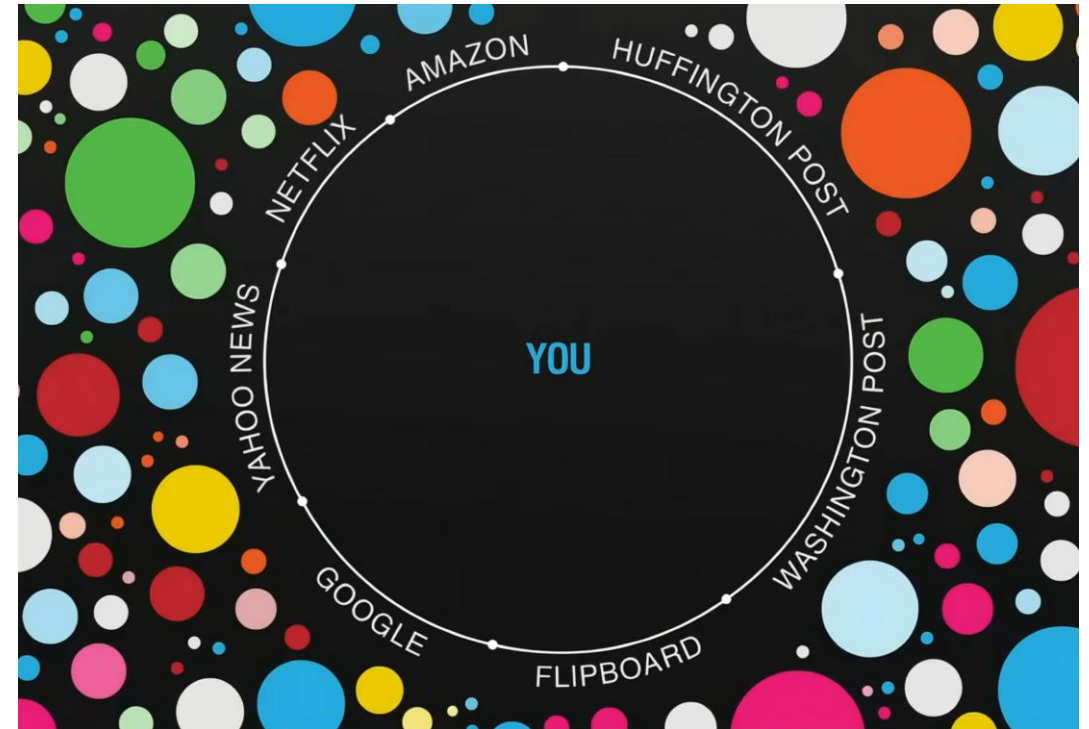
1. Biased система
2. Порождает biased данные
3. Обучение на которых создает еще более biased систему



Filter bubble

- Алгоритмы управляют тем, что видят пользователи
 - И формируют для них персонализированное представление реальности
- Цель алгоритмов – удержать внимание
- Вместо “сбалансированной информационной диеты”, пользователи получают фастфуд

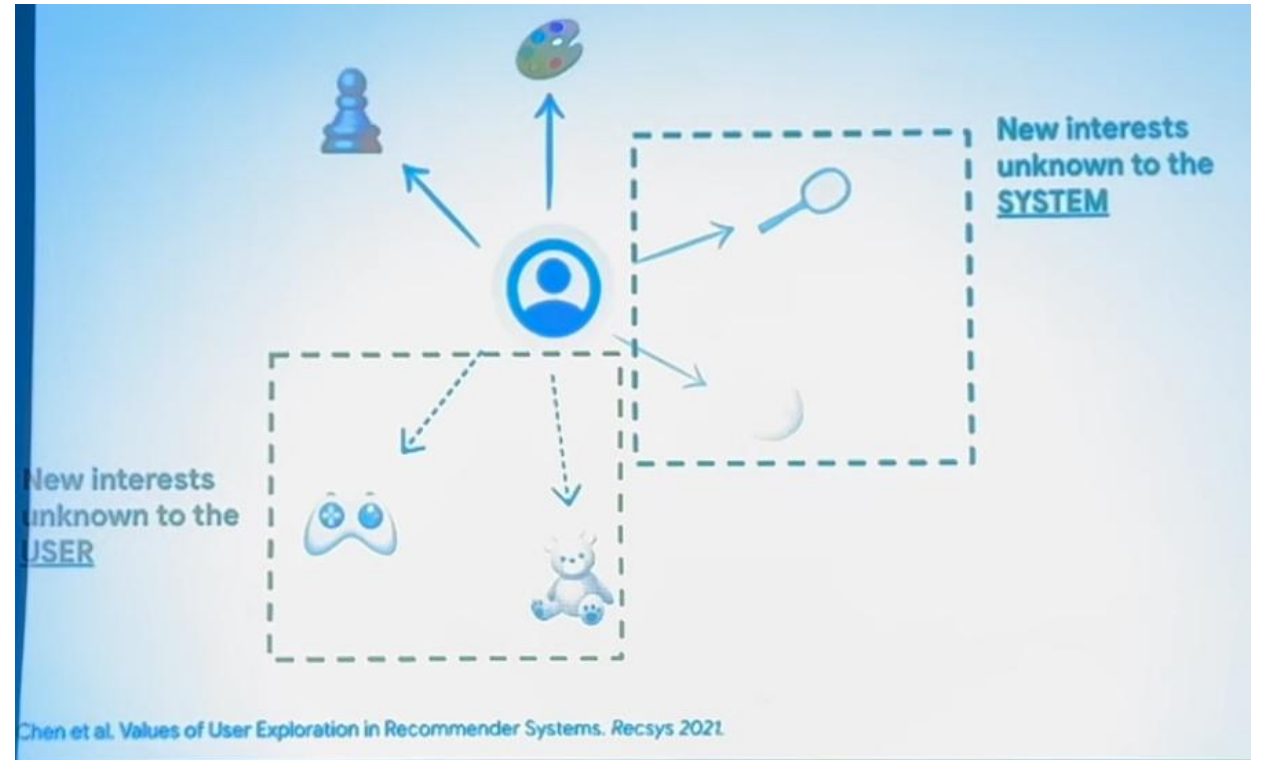
Echo chamber: в информационный пузырь попадает только информация, которая поддерживает позицию пользователя.



[Beware online "filter bubbles" | Eli Pariser](#)

Filter bubble

- Вне того, что рексистема привыкла показывать, есть контент, который понравился бы пользователю
- Как настроить систему так, чтобы она это всё равно нашла? Если нет фидбека



[User Intents and Journeys in Recommender Systems by Minmin Chen | VideoRecSys Workshop | RecSys 2023](#)

Popularity bias

- **Mathew effect:** айтемы, у которых уже часто рекомендуются, будут только чаще рекомендоваться; остальное – реже
 - Свежий и нишевый контент надо рекомендовать
- Есть хороший контент, который рексистема редко показывает
 - Как сделать так, чтобы она начала его чаще показывать? Если нет фидбека



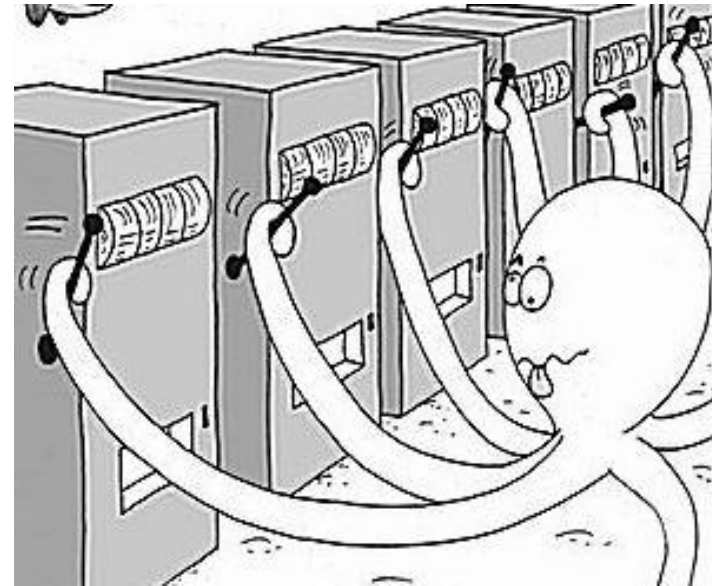
Exploration

- **User exploration:** хотим выходить за пределы пользовательского “информационного пузыря”, исследовать его интересы
 - Показывать что-то, что не входит в привычные для алгоритма и пользователя рекомендации
- **Item exploration:** хотим выходить за пределы популярных айтемов, исследовать весь каталог
 - Показывать что-то, что не входит в топы популярного по мнению алгоритма

Вместо “жадного” режима работы алгоритмов – нужна рандомизация.

Multi-armed bandits

- Однорукий бандит – автомат в казино
 - Дергаем ручки автоматов, в надежде получить выигрыш
- Многорукий бандит – сразу несколько ручек
- Задача: выиграть как можно больше денег



[Multi-Armed Bandits at Microsoft Research](#)

Бандиты в рекомендациях

- Показываем рекламные баннеры
- Какой баннер принесёт наибольшую прибыль?

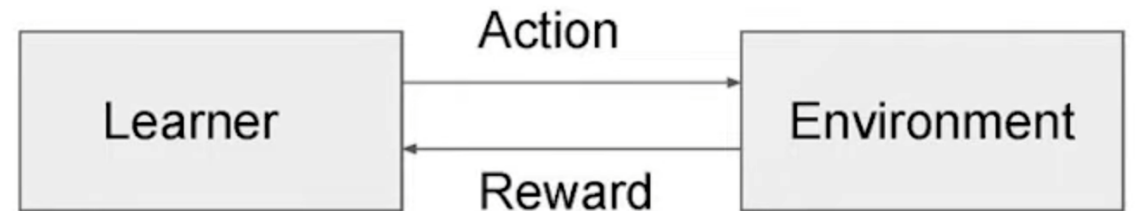


What is the most convincing ad
for donations to wildlife preservation?

[Tutorial - An industry perspective on Bandit Feedback](#)

Постановка задачи

- Агент выбирает действия; “дергает ручки”
- Среда – всё, что не зависит агента; внешние факторы
 - Среда задаёт множество возможных действий A
- В ответ на действия, среда даёт награду $r \in \mathbb{R}$
- У агента есть политика $\pi(a)$ – логика, по которой он действует; модель поведения
 - В общем случае она стохастическая



[A Multi-Armed Bandit Framework for Recommendations at Netflix | Netflix](#)

Постановка задачи

- $\{p(r \mid a), a \in A\}$ – набор распределений, задающий бандитную задачу
- $q(a) = \mathbb{E}_{p(r|a)} [r]$ – ожидаемая награда от ручки a
- $a^* = \operatorname{argmax}_a q(a)$ – оптимальная ручка
- $v^* = q(a^*)$ – выигрыш от оптимальной ручки

Цель бандитного алгоритма

- Пусть у нас конечный эпизод длины T и получаемые в эпизоде награды обозначаются как $r_1, \dots, r_T$. Тогда
 - Максимизируем выручку – кумулятивную награду

$$\mathbb{E} \left[\sum_{t=1}^T r_t \right] \rightarrow \max$$

- Минимизируем сожаление (regret) – разницу между выигрышем для оптимальной и нашей политики

$$\mathbb{E} \left[\sum_{t=1}^T (v^* - r_t) \right] \rightarrow \min$$

Обучение с подкреплением

- В общем обучении с подкреплением у среды есть состояние $s \in S$
- На шаге t :
 - Агент выбирает действие $a_t \sim \pi(a \mid s_t)$
 - Получает награду $r_t \sim p(r \mid s_t, a_t)$
 - И среда переходит в новое состояние $s_{t+1} \sim p(s_{t+1} \mid s_t, a_t)$
- Есть некоторая **динамика среды**, на которую влияет агент
- Если раньше у нас было одно оптимальное действие, то теперь для разных состояний - разные оптимальные действия

Контекстные бандиты

- Это шаг от бандитов в сторону общего RL
- На шаге t
 - Агент видит контекст $x_t \in \mathbb{R}^d$
 - Выбирает действие $a_t \sim \pi(a \mid x_t)$
 - Получает награду $r_t \sim p(r \mid x_t, a_t)$
- Но:
 - Контекст не зависит от действий агента
- Можно сделать отдельного бандита на каждый уникальный контекст, но только если их не очень много

Рекомендательная система

- Рексистема - агент

Рекомендательная система

- Рексистема - агент
- Пользователи, каталог, контекст рекомендации - среда
- Действия – рекомендации
 - Показываем по одному айтему (моя волна), $A = \mathcal{I}$
 - Или показываем цельную выдачу $A = \mathcal{I} \times \dots \times \mathcal{I}$
- Награда – фидбек от пользователя
 - E.g., $r \in \{0, 1\}$, был клик или нет
 - Или $r \in \mathbb{R}$, сумма заказа
 - Или даже сумма разных фидбеков
- Политика – модель поведения рексистемы
 - Наш рекомендательный алгоритм



Bandit algorithms

Часть 2

Greedy

- На каждом шаге выбираем действие с максимальной оценкой ожидания награды
 - $a_t = \operatorname{argmax} q_t(a)$
 - $q_t(a) = \frac{\sum_{l=1}^t \mathbb{1}\{a_l = a\}}{n_t(a)}$
- То есть поддерживаем и обновляем оценки мат ожиданий наград ручек
 - Инициализация: попробовать каждую ручку по одному разу

Какие недостатки алгоритма?

Greedy

- На каждом шаге выбираем действие с максимальной оценкой ожидания награды
 - $a_t = \operatorname{argmax} q_t(a)$
 - $q_t(a) = \frac{\sum_{l=1}^t \mathbb{1}\{a_l = a\}}{n_t(a)}$
- То есть поддерживаем и обновляем оценки мат ожиданий наград ручек
 - Инициализация: попробовать каждую ручку по одному разу

Какие недостатки алгоритма? Из-за шума можем зафиксироваться на плохой ручке.

Exploration-exploitation dilemma

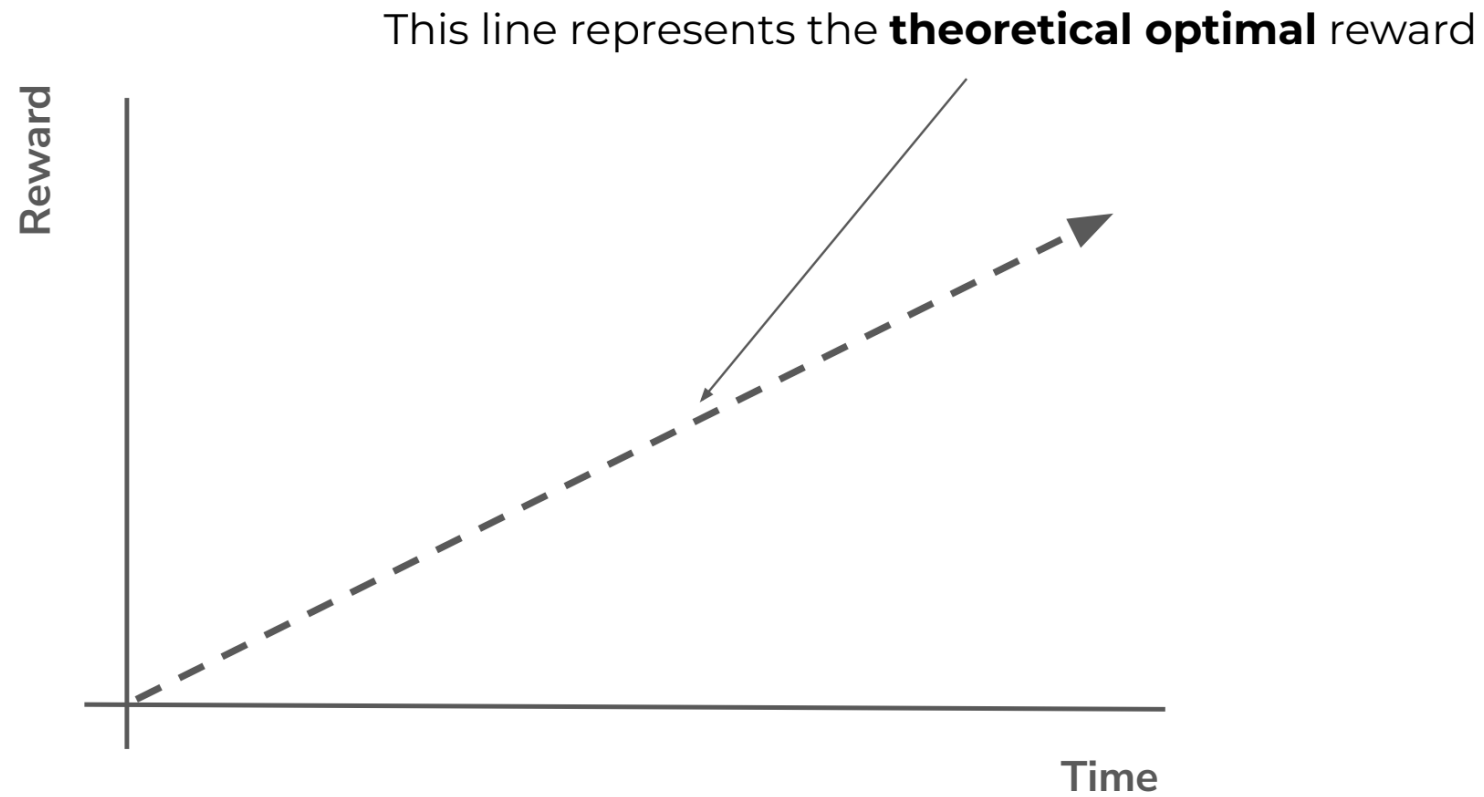
- **Exploitation:** используем текущие знания и выбираем действия с максимальной ожидаемой наградой
- **Exploration:** пробуем альтернативные действия, чтобы улучшить оценки наград
- **Дилемма:**
 - Без exploration можно застрять на плохом действии
 - Без exploitation теряем награду
- Цель бандитного алгоритма – найти такой баланс exploration и exploitation, при котором получим максимум награды
 - и минимум сожалений :)

Explore-then-commit

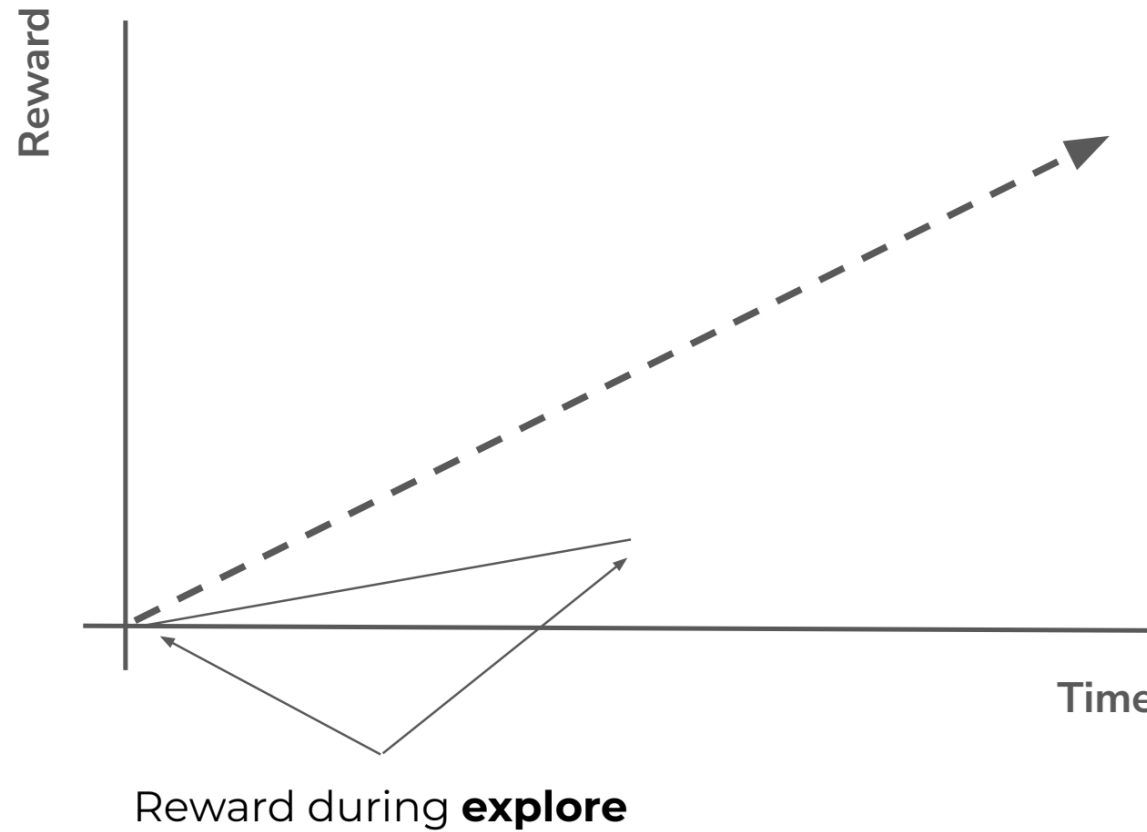
- Фаза exploration: сначала много раз пробуем все действия, чтобы оценить их ожидаемые награды
- Фаза exploitation: фиксируем лучшее действие и дальше всегда его выбираем

Нужно заранее выбрать длительность exploration фазы – если слишком короткая, то можем недоисследовать, слишком длинная – потерять награду.

Explore-then-commit

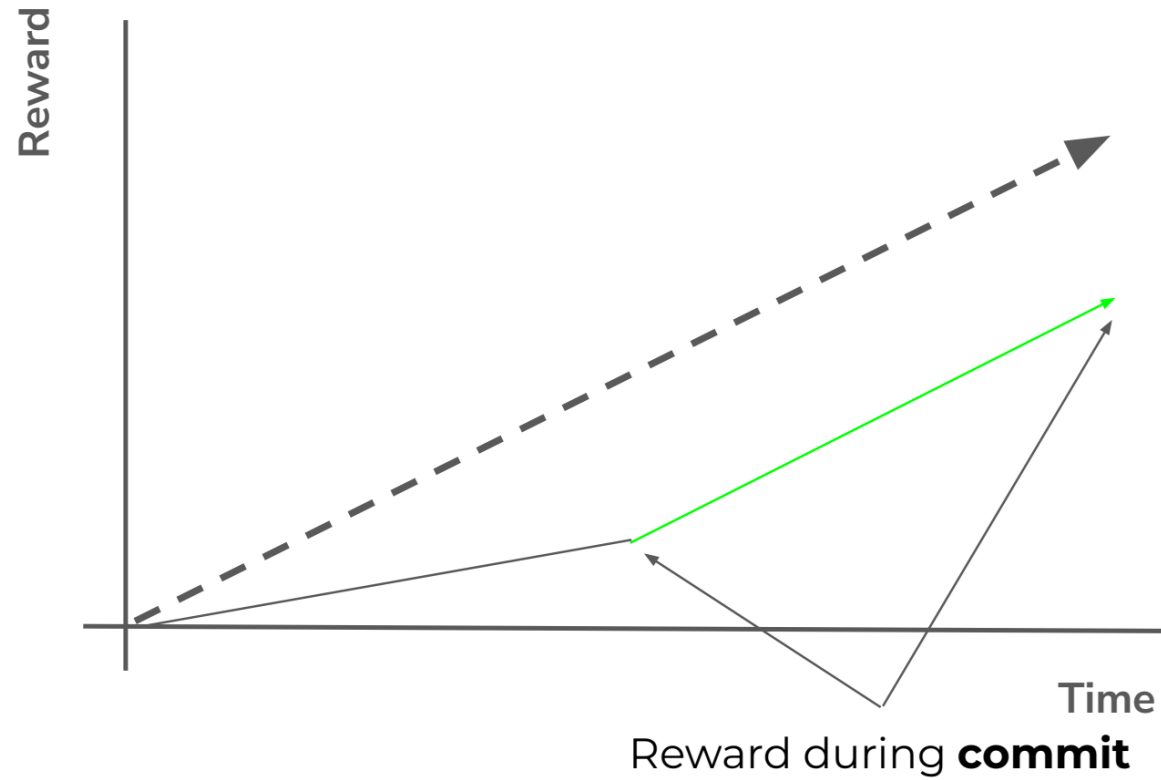


Explore-then-commit

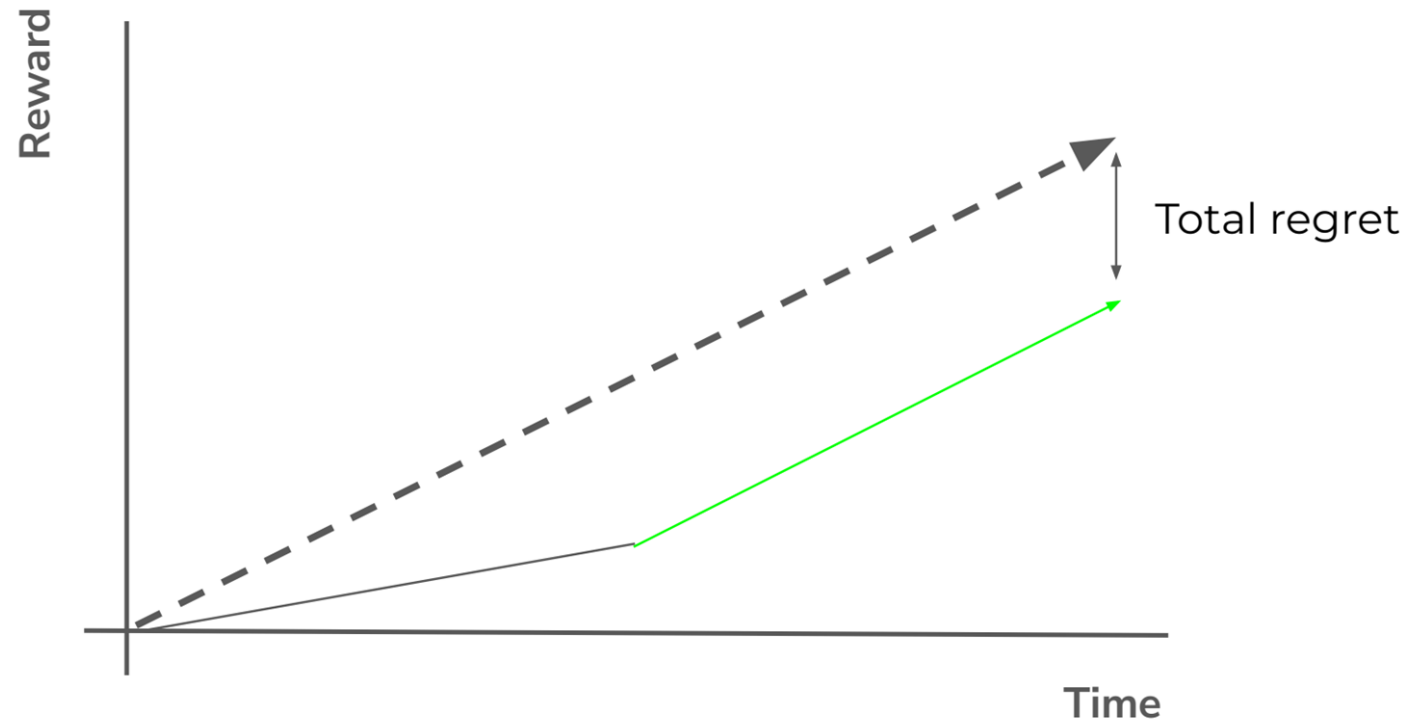


[Tutorial - An industry perspective on Bandit Feedback](#)

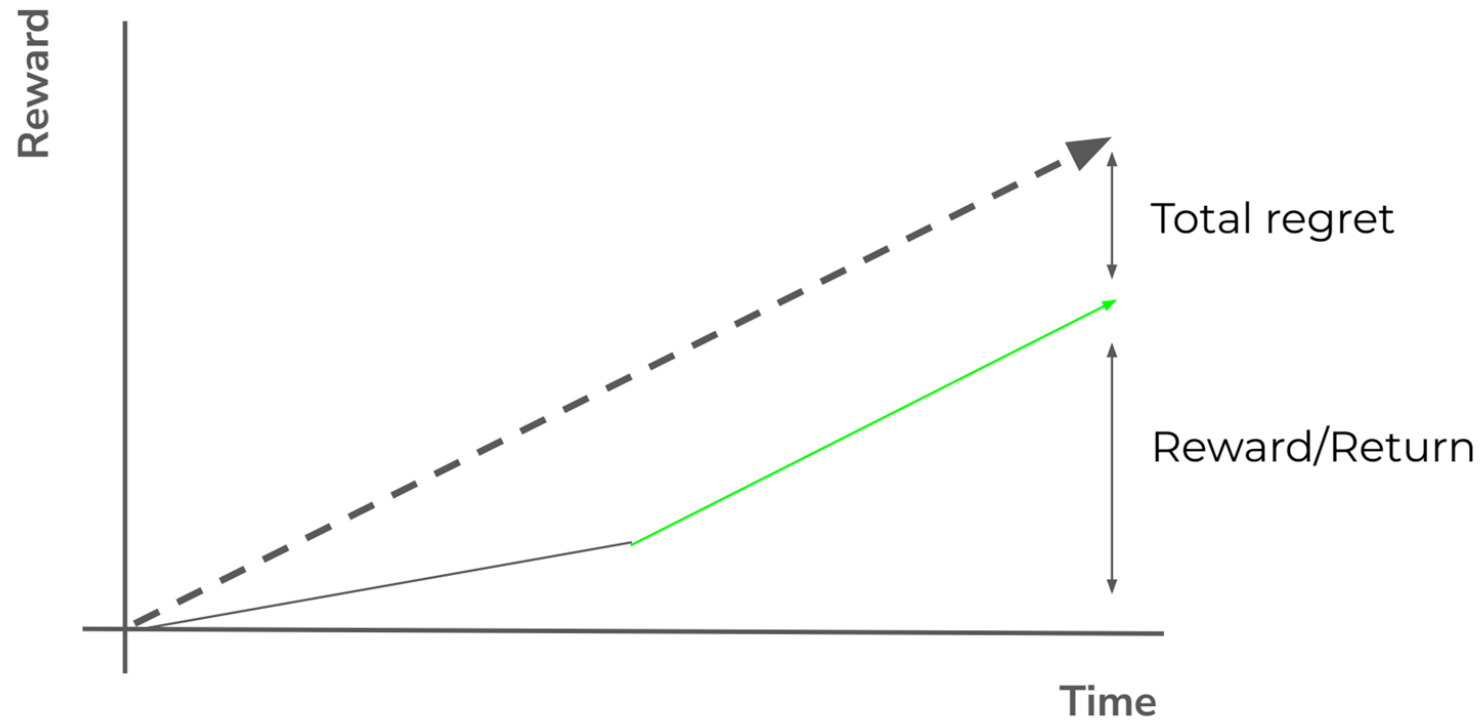
Explore-then-commit



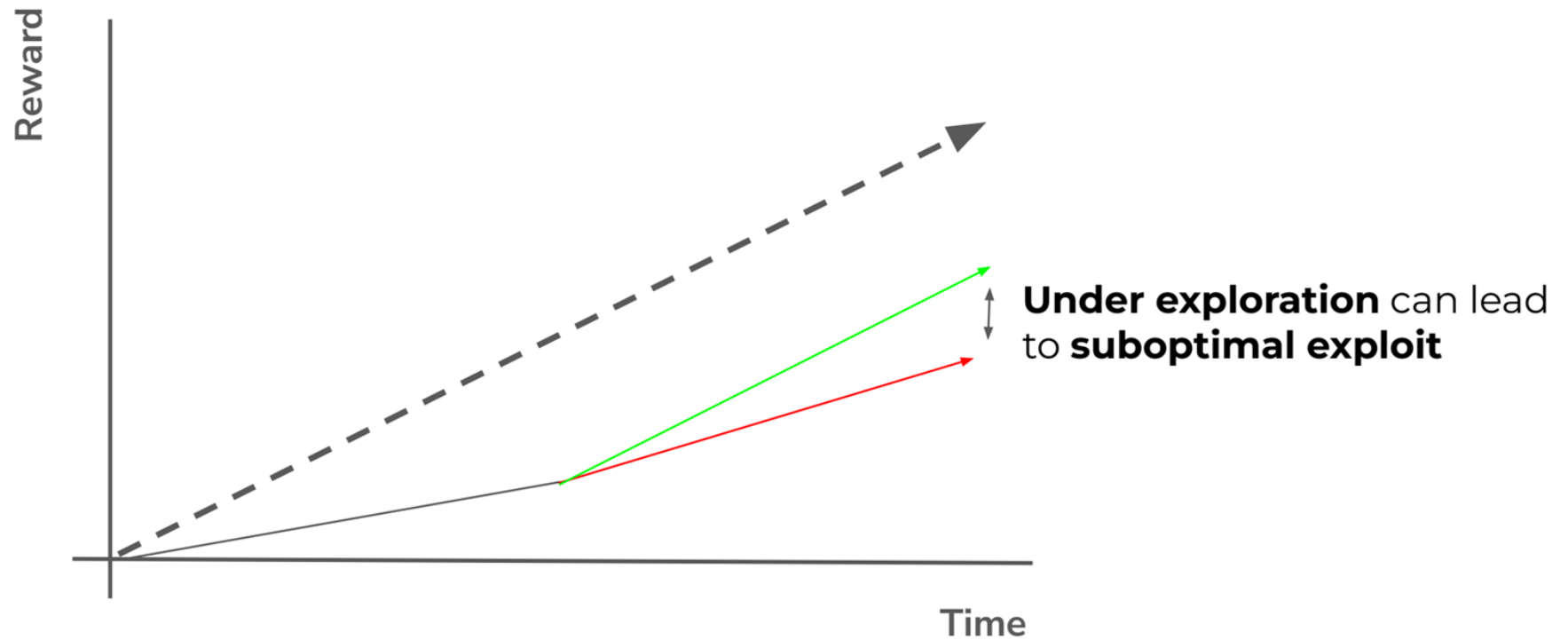
Explore-then-commit



Explore-then-commit



Explore-then-commit



Epsilon-greedy

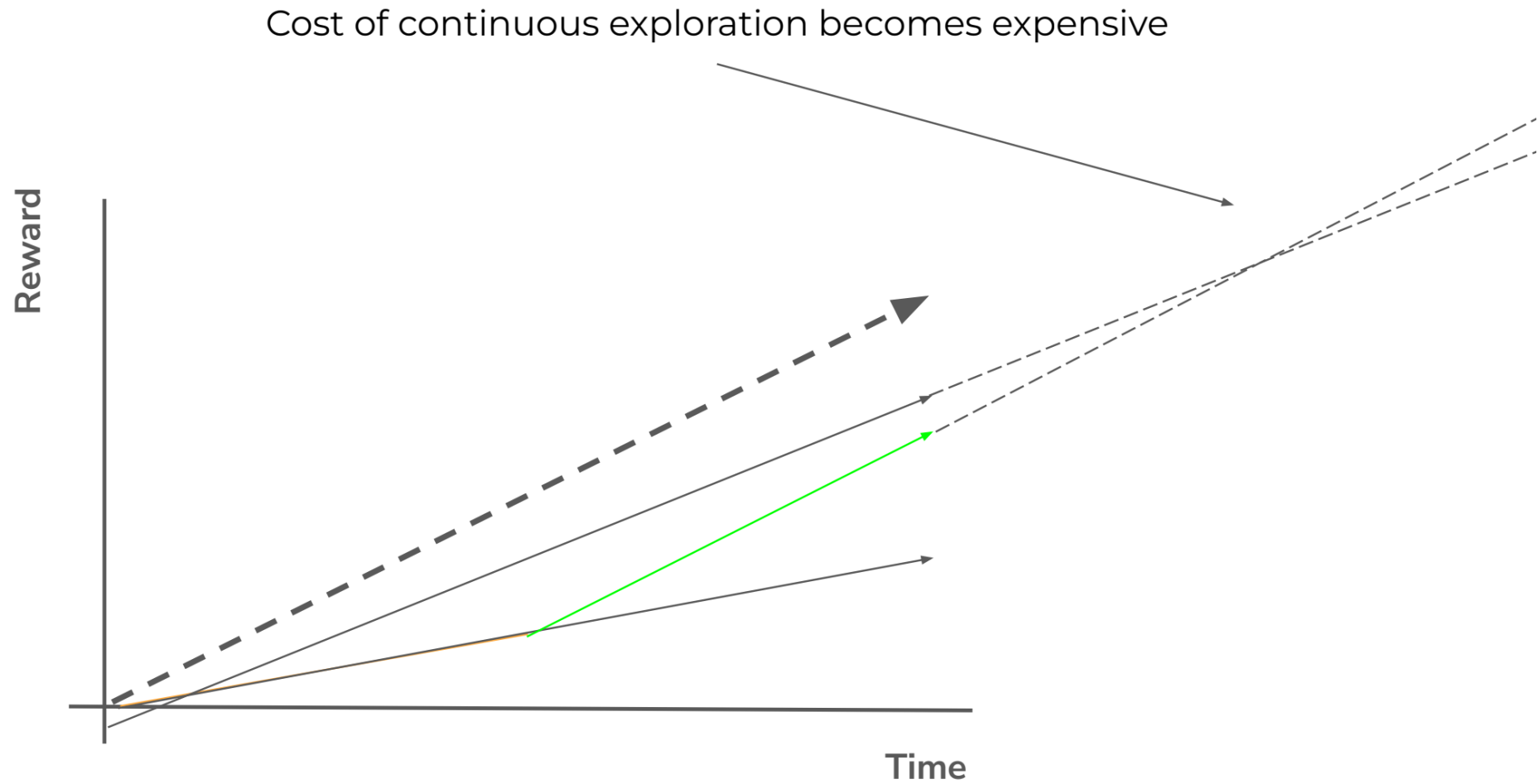
- С вероятностью $1 - \epsilon$ выбираем действие с максимальной оценкой награды
- С вероятностью ϵ действуем случайно

$$\pi(a_t = a) = (1 - \epsilon) \mathbb{1} \{a = a^*\} + \frac{\epsilon}{|A|}$$

Недостатки

- Неоптимальный exploration: Во время исследований продолжаем выбирать заведомо плохие действия

Epsilon-greedy



Epsilon-greedy

Можно со временем уменьшать epsilon

Какие минусы?

Epsilon-greedy

Можно со временем уменьшать epsilon

Какие минусы?

- Плохо адаптируется к изменяющейся среде (к distribution drift)
- Всё еще неоптимальная фаза исследования
- Нужно заранее правильно выбрать скорость уменьшения epsilon



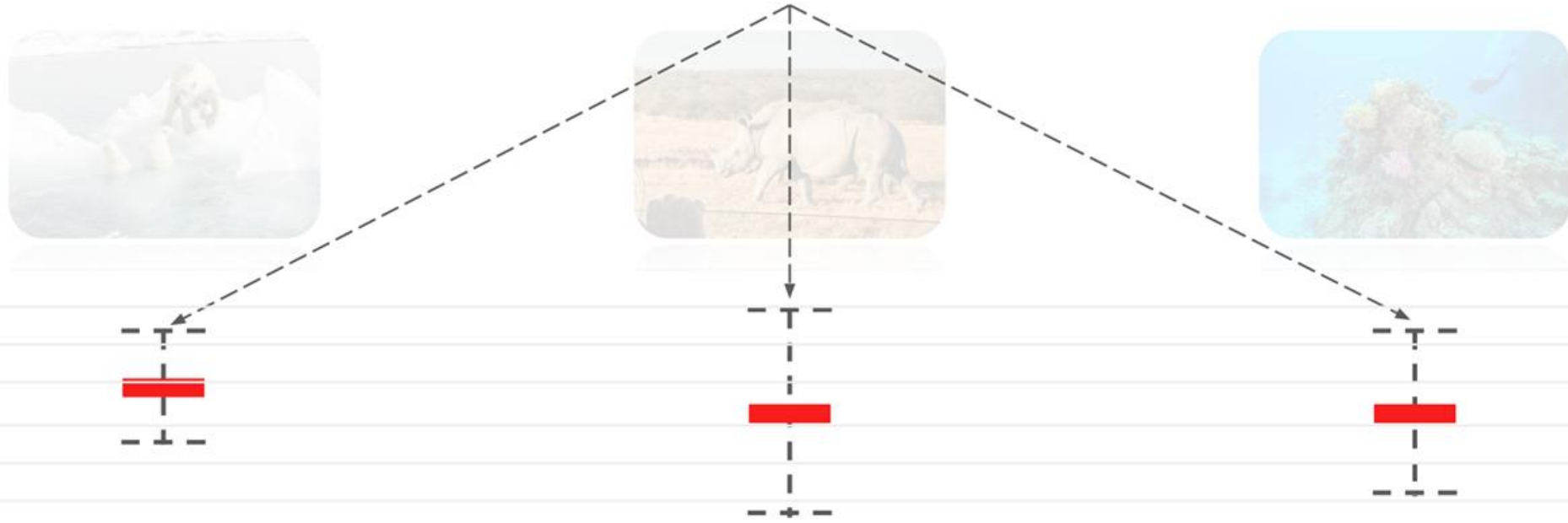
How can we do **better than uniform random** exploration?

Upper confidence bound (UCB)



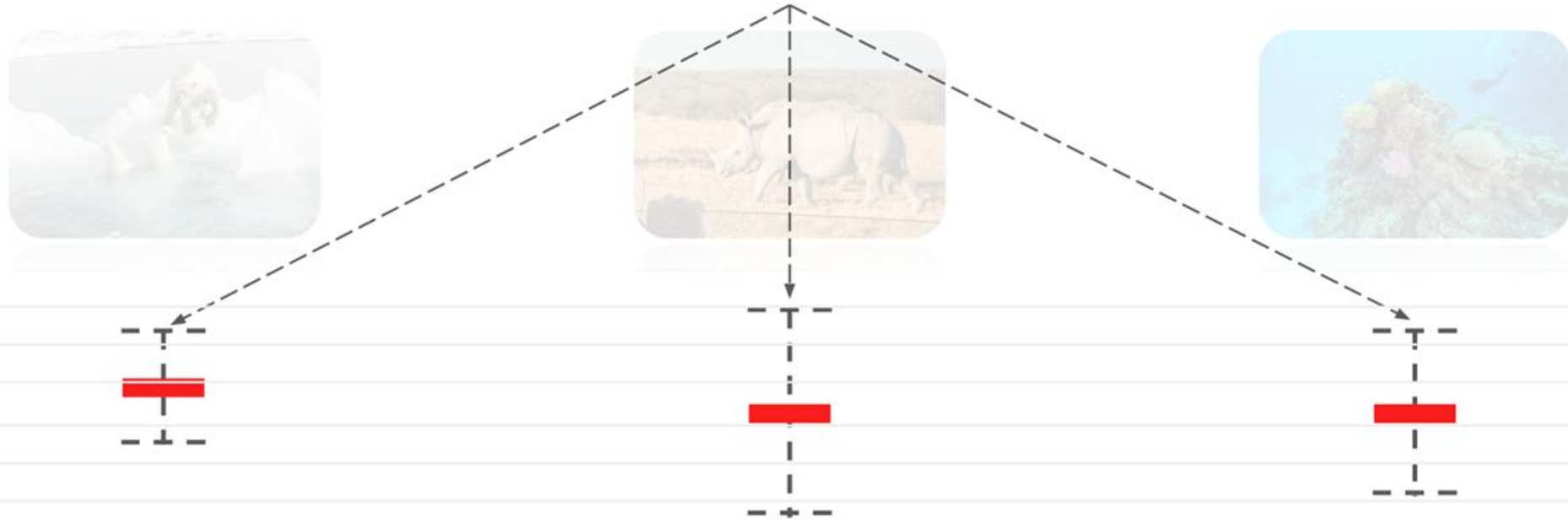
- Идея: optimism in face of uncertainty
 - Выбираем действие с наибольшей верхней оценкой мат ожидания награды
- $$a_t = \operatorname{argmax}_a [q_t(a) + u_t(a)]$$
 - $q_t(a)$ – оценка мат ожидания награды
 - $u_t(a)$ – бонус за неопределенность

UCB



UCB converts each arm into confidence bounds

UCB



UCB converts each arm into confidence bounds

$$a_t = \operatorname{argmax}_a \left[q_t(a) + c \sqrt{\frac{\log t}{n_t(a)}} \right]$$

UCB: вывод

Теорема 78 — Неравенство Хёфдинга: Пусть $X_1 \dots X_n$ — i.i.d выборка из распределения на домене^a $[0, 1]$ с истинным средним μ , $\hat{\mu} := \frac{1}{N} \sum_i^N X_i$ — выборочная оценка среднего. Тогда $\forall u > 0$:

$$\mathbf{P}(\mu \geq \hat{\mu} + u) \leq e^{-2nu^2}$$

Утверждение 78: Для любого δ с вероятностью $1 - \delta$ истинное значение $Q(a)$ не превосходит $Q_k(a) + U_k(a)$, где:

$$U_k(a) := \sqrt{\frac{-\ln \delta}{2n_k(a)}}$$

Доказательство. В силу неравенства Хёфдинга:

$$\mathbf{P}(Q(a) \geq Q_k(a) + U_k(a)) \leq \exp^{-2n_k(a)U_k(a)^2}$$

Возьмём заданное δ и прогарантируем, что $\exp^{-2n_k(a)U_k(a)^2} = \delta$. Разрешая это равенство относительно $U_k(a)$, получаем доказываемое. ■

В качестве δ на k -ом шаге будем выбирать, например, $\frac{1}{k^c}$, где c — гиперпараметр. Таким образом, истинное значение будет всё с большей вероятностью оказываться внутри доверительного интервала. Итого в алгоритме UCB предлагается следующая формула выбора действия на очередном k -ом шаге:

$$a_k = \operatorname{argmax}_a \left[Q_k(a) + c \sqrt{\frac{\log k}{n_k(a)}} \right] \quad (7.3)$$

Thompson sampling

- Байесовский подход к задаче бандитов
- Для каждого действия a задаём модель награды: $r \sim p_\theta(r \mid a)$
- Также задаём распределение над параметрами: $\theta \sim p(\theta)$
- Probability matching - выбираем действие пропорционально вероятности того, что оно оптимально:

$$\pi(a) = p(a = a^*)$$

- На практике:
 - Сэмплируем параметры: $\theta \sim p(\theta \mid D)$
 - выбираем действие: $a_t = \operatorname{argmax}_a \mathbb{E}_{p_\theta(r|a)} [r]$
- Неопределенность в параметрах – exploration

Beta-Bernoulli bandit

- Награда бинарная: $r \sim p_{\theta}(r \mid a) = \text{Bernoulli}(\theta_a)$
- Априорное распределение:
 - $\theta_a \sim \text{Beta}(\alpha_a, \beta_a)$
- Апостериорное распределение (обновление параметров, после того как мы пронаблюдали награду):
 - $(\alpha_a, \beta_a) \rightarrow (\alpha_a + r, \beta_a + 1 - r)$
- Thompson sampling:
 - $\theta_a \sim \text{Beta}(\alpha_a, \beta_a)$
 - $a_t = \underset{a}{\operatorname{argmax}} \theta_a$

Beta-Bernoulli bandit

Алгоритм 26: Beta-Bernoulli Бандит с сэмплированием Топмсона

Гиперпараметры: α, β — параметры прайора.

Обнулить счётчики $\alpha_0(a) := \alpha, \beta_0(a) := \beta$

На очередном шаге k :

1. сгенерировать $\theta_a \sim \text{Beta}(\alpha_k(a), \beta_k(a))$
2. выбрать $a_k := \underset{a}{\operatorname{argmax}} \theta_a$
3. пронаблюдать $r_k \in \{0, 1\}$
4. обновить счётчик $\alpha_k(a) := \alpha_{k-1}(a) + [a_k = a][r_k = 1]$
5. обновить счётчик $\beta_k(a) := \beta_{k-1}(a) + [a_k = a][r_k = 0]$

Визуализация

<https://learnforeverlearn.com/bandits/>

Practical considerations

Часть 3

Контекстные бандиты

Контекст может включать:

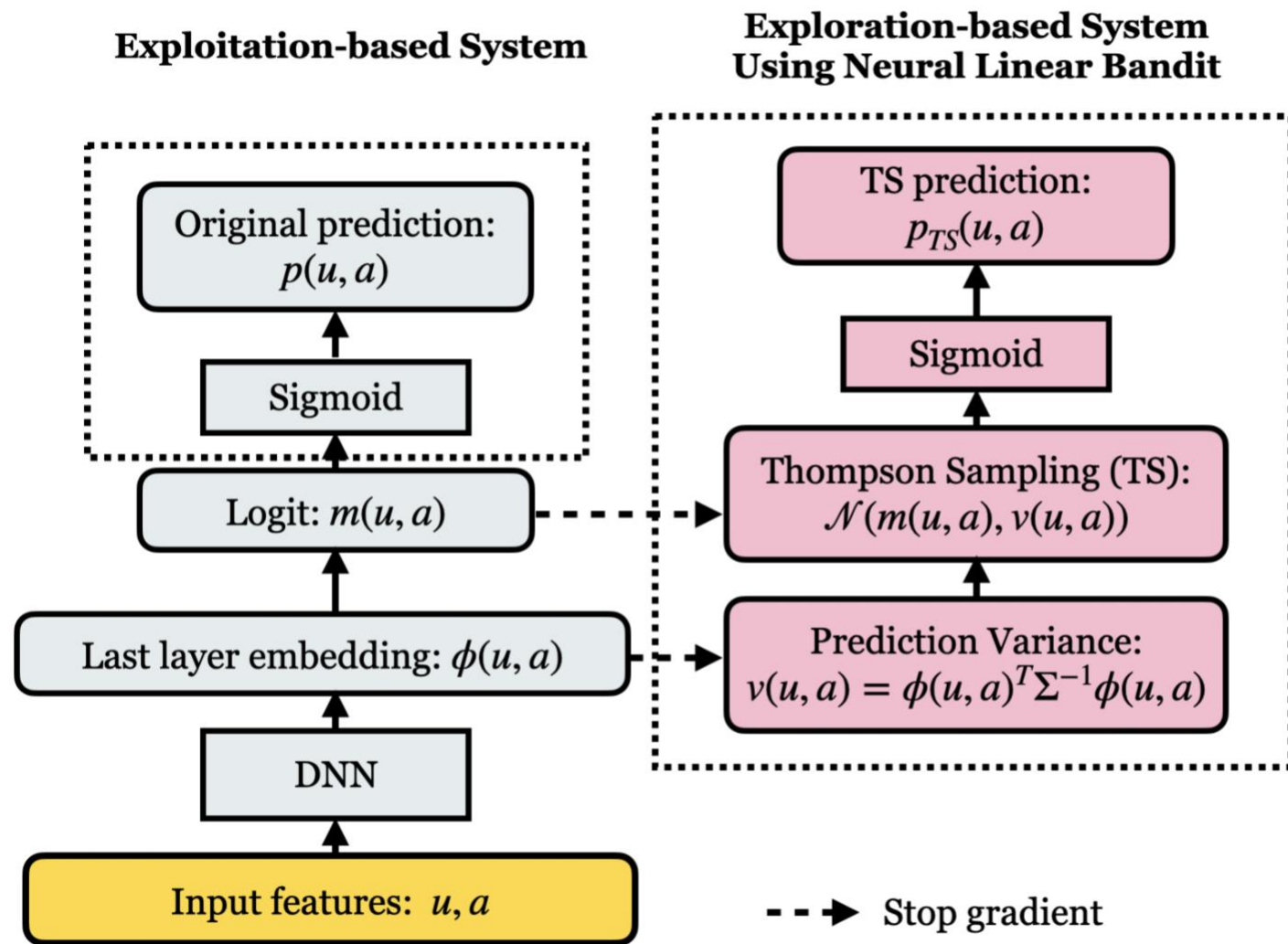
- Информацию о пользователе
- Время, устройство, локацию
- Arm context – признаки айтема

Линейные контекстные бандиты

- Линейная модель $r = x^T \theta^* + \epsilon$
 - x – контекст
 - θ^* - неизвестные параметры
- linUCB – подбираем θ и оцениваем неопределенность, получаем формулу вида
 - $$a_t = \operatorname{argmax}_a \left[x_{t,a}^T \theta_a + \alpha \sqrt{x_{t,a}^T A_a^{-1} x_{t,a}} \right]$$
 - $$A_a = \lambda I + \sum_{t=1}^T x_{t,a} x_{t,a}^T$$
- linTS: байесовская линейная регрессия

Case study: YouTube ranking

- Используют Neural Linear Bandit
- Байесовская регрессия поверх вектора из последнего скрытого слоя



Оценка качества

- **Онлайн:**
 - A/B тестирование
 - Важно учитывать warmup период, когда алгоритм ещё учится
 - Мало кто умеет замерять влияние на данные
- **Оффлайн**
 - Оценка сложна из-за того, что залогированные данные собраны из другой политики
 - Replay – простой метод, но требует полностью рандомизированного трафика
 - Сложный способ – off-policy evaluation

Онлайн-обновление системы

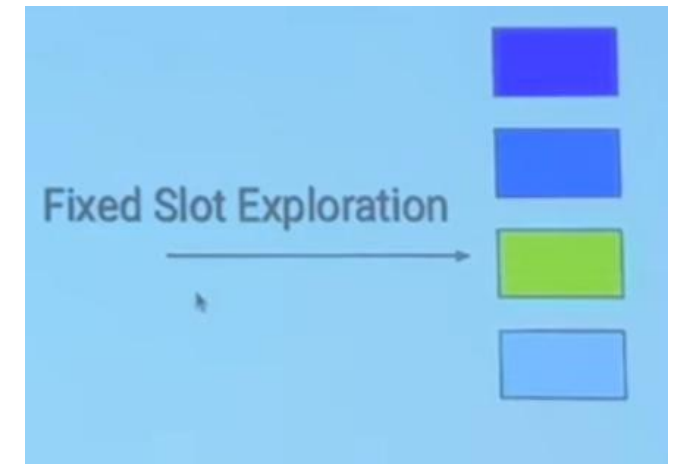
- В теории, нужно обновлять алгоритм после каждого взаимодействия
- На практике так делать не можем – много асинхронно обрабатываемых запросов от пользователей
 - Обновления делаем батчами – ждем пока накопится определенное количество взаимодействий и затем обновляем
- Данные для обучения собираются текущей политикой – это on-policy RL

Distribution drift

- В реальности распределение наград меняется со временем
- Классические бандитные алгоритмы предполагают стационарность
 - Пример: из-за сезонности одна ручка временно стала лучше, алгоритм её переоценил и продолжал долго выбирать
- Но их можно адаптировать
 - Постоянный ϵ
 - Экспоненциальное сглаживание для оценки средней награды

Exploration на практике

- **Fixed slot exploration** – фиксируем позицию в выдаче и используем её для исследования
- **Boltzman (softmax) exploration** – $\pi(a) \propto \exp(\tau s(a))$
 - Если слишком много айтемов, то сначала оставляем Top-K
 - С помощью температуры регулируем степень exploration
- Перемешивание соседних позиций в выдаче
- YouTube использует контекстных бандитов
 - для ранжирования Neural Linear Bandit
 - для генерации кандидатов модификацию LinUCB



[Youtube Discovery Evolution by Lukasz Heldt | VideoRecSys Workshop Keynote | RecSys 2023](#)

Practical considerations

- Нужно кэшировать результаты рандомизированной выдачи
- Бизнес-правила поверх политики усложняют оценку
- Многозадачные бандиты

Особенности рексистем

- Масштаб
 - Очень большие пространства действий
 - Гигантские каталоги
 - Богатые контексты
- Ограниченный exploration
 - Нельзя сильно ухудшать пользовательский опыт
- Инженерные ограничения – сложно делать realtime обновления
- Действия системы влияют на пользователей и каталог
 - Бандиты – это упрощение

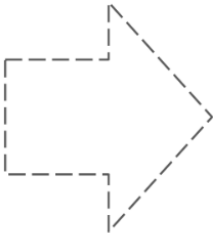
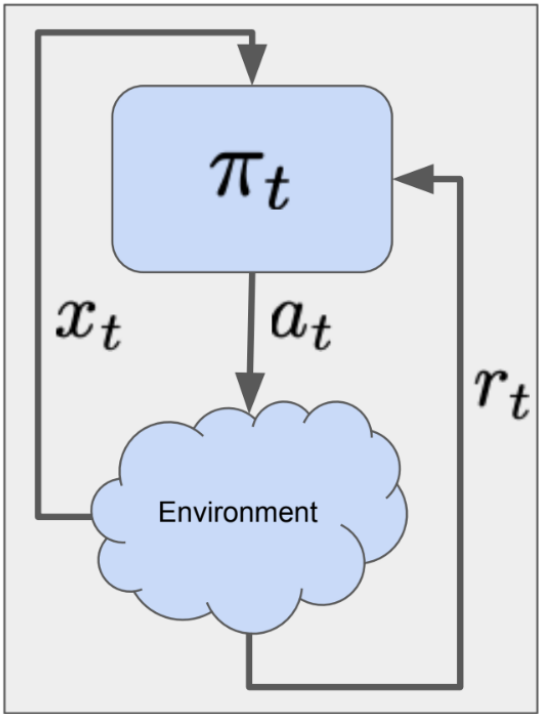
Summary

- Обсудили зачем нужен exploration
 - Feedback loop, exposure bias, filter bubbles, popularity bias
- Рассмотрели постановку задачи бандитов
- Обсудили алгоритмы: UCB, Thompson Sampling, их модификации для контекстных бандитов

Beyond on-policy bandits

Часть 4

Off-policy evaluation (OPE)



			
	1		
		0	
		1	
			0
		0	
			1
	1		
		0	
			0

Пусть у нас есть залогированные данные из другой политики

Off-policy evaluation

- Хотим оценить качество новой политики

$$\mathbb{E}_{a \sim \pi} \left[\sum_{t=1}^T r_t \right]$$

- У нас есть только сэмплы из старой политики π_0
- Эта фундаментальная проблема оффлайн-оценки качества рексистем
 - Пытаемся оценивать новые модели и алгоритмы на данных, залогированных при другой продовой политике

Direct method

Directly model the reward for a given context-action pair:

$$r_{\theta}(x,a) \approx E[R \mid X, A]$$

And **estimate the counterfactual rewards**

			
	1	1	1
	0	0	0
	0	1	0
	1	0	0
	0	0	1

$$V(\pi, D_0) = \sum_i r_{\theta}(x, \pi(x))$$

Inversed propensity scoring (IPS)

Inverse Propensity Scoring (**IPS**):

“How much more or less likely is the new policy to perform actions that led to rewards in the training data?”

$$V(\pi, D_0) = \sum_i r_i * \pi(a|x_i) / \pi_0(a|x_i)$$

Off-policy evaluation

- Direct method – high bias, low variance
- IPS – no bias, high variance
- Doubly Robust Estimator = DM + IPS
 - No bias, lower variance
- Можно делать off-policy learning –
градиентный спуск по полученному в
OPE выражению

Full RL formulation

- Полная постановка обучения с подкреплением MDP, (S, A, P, R, π) :
 - S – множество состояний среды
 - A – множество действий, доступных агенту
 - $P(s_{t+1}|s_t, a)$ – переходные вероятности
 - Прямое влияние агента на состояние среды
 - $p(r | s, a)$ – награда, выдаваемая агенту в ответ на действия
 - $\pi(a | s)$ – политика агента
 - $\tau = (s_1, a_1, r_1, s_2, a_2, r_2, \dots)$ – траектория (эпизод)

$$V(\pi) = \mathbb{E}_{s_1 \sim \rho(s), a_t \sim \pi(a|s_t), s_{t+1} \sim p(s|s_t, a_t)} \left[\sum_{t=1}^T \gamma^t r_t \right] = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)]$$

REINFORCE

- Пусть у нас параметризованная политика $\pi_{\theta}(a | s)$
- Хотим максимизировать кумулятивную награду

$$V(\pi) \rightarrow \max_{\theta}$$

- Нужно посчитать градиент по параметрам:

$$\nabla_{\theta} V(\pi) = \nabla_{\theta} \mathbb{E}_{\pi_{\theta}} [R(\tau)]$$

Log-derivative trick

$$\begin{aligned}\nabla_{\theta} V(\pi) &= \nabla_{\theta} \mathbb{E}_{\pi_{\theta}} [R(\tau)] = \mathbb{E} [\nabla_{\theta} \pi_{\theta}(\tau) R(\tau)] = \\ &= \{ \nabla_{\theta} \pi_{\theta} = \pi_{\theta} \nabla_{\theta} \log \pi_{\theta} \} = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(\tau) R(\tau)] = \\ &= \mathbb{E}_{\pi_{\theta}} \left[\left(\sum_{t=1}^T \nabla_{\theta} \log \pi(a_t | s_t) \right) R(\tau) \right] \approx \sum_{\tau \sim \pi_{\theta}(\tau)} \sum_{t=1}^T \nabla_{\theta} \log \pi(a_t | s_t) R(\tau)\end{aligned}$$

- Максимизируем вероятность действий, приводящих к хорошей награде
- Это on-policy алгоритм
 - обновляем параметры политики, с помощью которой сэмплилировали траектории

Off-policy correction

$$\begin{aligned}\mathbb{E}_{\pi_{\theta}} \left[\left(\sum_{t=1}^T \nabla_{\theta} \log \pi(a_t | s_t) \right) R(\tau) \right] &= \mathbb{E}_{\pi_0} \left[\frac{\pi_{\theta}(\tau)}{\pi_0(\tau)} \left(\sum_{t=1}^T \nabla_{\theta} \log \pi(a_t | s_t) \right) R(\tau) \right] = \\ &= \mathbb{E}_{\pi_0} \left[\frac{\prod_{t=1}^T \pi_{\theta}(a_t | s_t)}{\prod_{t=1}^T \pi_0(a_t | s_t)} \left(\sum_{t=1}^T \nabla_{\theta} \log \pi(a_t | s_t) \right) R(\tau) \right]\end{aligned}$$

- Оцениваем градиент с помощью importance sampling
- Как и в бандитах, здесь high variance

Case study: YouTube REINFORCE Recommender System

- Взяли двухбашенную модель для генерации кандидатов (RNN, предсказывающая следующий просмотр пользователя)
- Заменяли sampled softmax loss на REINFORCE
 - Долгосрочная награда – время просмотра на горизонте 4-10 часов
 - REINFORCE = sampled softmax loss, но со взвешиванием сэмплов
 - Веса = кумулятивная награда на траектории
- Off-policy correction: упрощают importance weight до $\pi_{\theta}(a_t | s_t) / \pi_0(a_t | s_t)$

Summary

- Обсудили зачем нужен exploration
 - Feedback loop, exposure bias, filter bubbles, popularity bias
- Рассмотрели постановку задачи бандитов
- Обсудили алгоритмы: UCB, Thompson Sampling, их модификации для контекстных бандитов
- Поговорили про off-policy evaluation
- Затронули общую RL постановку рексистем и метод REINFORCE

На семинаре – применение RL для Яндекс Лавки.